

JJ MultiSet System Guide

Item	Value
Document Title	JJ MultiSet System Guide
Project	JJ_MultiSet
Document Type	System Guide
Version	Draft 1.0
Last Updated	2026-04-20
Primary Scope	Architecture, runtime flow, integration, and extension guidance

Table of Contents

- 1. [Purpose](#)
 - 2. [System Overview](#)
 - 3. [Architectural Overview](#)
 - 4. [Major Modules](#)
 - 1. [External Map Service Integration](#)
 - 2. [UI Workflow Controller](#)
 - 3. [Map Retrieval, Caching, and Database Import](#)
 - 4. [Editor Core](#)
 - 5. [Simulation Subsystem](#)
 - 6. [AI Guidance Subsystem](#)
 - 7. [Firebase Upload Subsystem](#)
 - 5. [End-to-End Workflow](#)
 - 6. [Data Model](#)
 - 1. [ExternalMapInfo](#)
 - 2. [MarkStorage.MarkData](#)
 - 3. [Firebase Example Structure](#)
 - 7. [Configuration](#)
 - 1. [Map Provider Configuration](#)
 - 2. [Firebase Configuration](#)
 - 3. [AI Configuration](#)
 - 8. [Extension Strategy](#)
 - 9. [Appendix](#)
 - 10. [Conclusion](#)
-

1. Purpose

This document describes the current system design, architecture, runtime flow, data model, integration points, and operational behavior of the JJ_MultiSet project.

It is intended for:

- software engineers maintaining or extending the project
- technical leads reviewing system structure and integration boundaries
- QA or implementation teams validating feature behavior
- project handoff and delivery documentation

This guide focuses on the software system as it exists in the current codebase, including:

- external map service integration
- map download and caching
- marker editing and simulation
- Firebase-based marker persistence
- AI agent text generation and speech playback

Suggested figure placeholders

Figure 1-1. End-to-end product workflow

Figure 1-2. High-level subsystem overview

2. System Overview

2.1 Product Summary

JJ_MultiSet is a Unity-based application that combines:

- external VPS/map-platform integration
- a spatial marker authoring workflow
- simulation-based user-position validation
- AI-generated guidance and speech output

The primary use case is to load a 3D map from an external provider, author semantic marker regions on top of that map, simulate user movement relative to those regions, and generate spoken guidance when the simulated context matches a marker.

2.2 Core Capabilities

The current implementation supports the following capabilities:

1. Authenticate against an external map platform.
2. Query and display available maps.

3. Download and load GLB map assets into Unity.
4. Cache downloaded maps locally.
5. Create and edit marker regions on the loaded map.
6. Configure marker geometry, directional constraints, keywords, and descriptive metadata.
7. Simulate user position and heading in relation to authored markers.
8. Upload marker datasets to Firebase Realtime Database.
9. Reload saved marker datasets from Firebase for a selected map.
10. Generate AI guidance text and play audio output through local or cloud TTS.

2.3 Architectural Intent

The project already contains two meaningful abstraction points:

- `IExternalMapService` isolates third-party map providers from the UI and map-loading flow.
- `IAgentTextService` and `IAgentSpeechService` isolate AI provider implementation from the simulation trigger logic.

These abstractions reduce coupling and create a viable path for future platform substitution.

3. Architectural Overview

3.1 Layered View

The system can be understood in five layers:

- 1. External Integration Layer**
Handles third-party map provider authentication, map listing, and download URL resolution.
- 2. Map Acquisition and Persistence Layer**
Handles GLB retrieval, local cache management, Firebase read/write operations, and map metadata storage.
- 3. Authoring and Simulation Layer**
Handles marker creation, editing, visualization, simulation, and state evaluation.
- 4. AI Guidance Layer**
Handles prompt construction, text generation, speech synthesis, and playback orchestration.
- 5. Presentation and Workflow Layer**
Handles Unity UI panels, user workflow transitions, and editor mode switching.

3.2 High-Level Component Map

CustomUIManager

- > IExternalMapService
- > MapQuery
- > CameraManager

MapQuery

- > IExternalMapService
- > Firebase Realtime Database
- > EditorManager

EditorManager

- > MarkerService
- > PanelController
- > SimulationController
- > UploadDB

SimulationController

- > MarkStorage
- > EditorManager UI output fields

AgentController

- > EditorManager
- > IAgentTextService
- > IAgentSpeechService

3.3 Runtime Responsibility Split

At runtime, the main orchestration path is:

1. CustomUIManager authenticates and presents maps.
2. MapQuery downloads and loads the selected map.
3. EditorManager manages authoring, simulation, and upload workflows.
4. SimulationController computes the current user-to-marker context.
5. AgentController observes simulation state and triggers AI guidance generation.

Suggested figure placeholders

Figure 3-1. Layered architecture diagram

Figure 3-2. Runtime interaction diagram

4. Major Modules

4.1 External Map Service Integration

Source files:

- `Assets/Script/Interface/IExternalMapService.cs`
- `Assets/Script/Interface/MultiSetMapService.cs`
- `Assets/Script/Interface/ExampleMapService.cs`

4.1.1 Responsibility

`IExternalMapService` defines the provider contract used by the rest of the application. The application depends on normalized map metadata rather than provider-specific API response structures.

4.1.2 Interface Contract

The interface defines three operations:

1. `AuthenticateAsync(...)`
2. `GetMapsAsync(...)`
3. `ResolveMapDownloadUrlAsync(...)`

This allows the UI and download logic to remain unchanged when switching map platforms.

4.1.3 Current Implementations

MultiSetMapService

Production-oriented integration for `api.multiset.ai`:

- authenticates using machine-to-machine credentials
- retrieves map lists from the remote API
- converts provider-specific map payloads into `ExternalMapInfo`
- resolves downloadable mesh URLs for downstream loading

ExampleMapService

A template implementation intended as a reference when integrating additional providers.

4.1.4 Engineering Note

This is one of the strongest abstraction boundaries in the current codebase. New provider integrations should be added by implementing `IExternalMapService` rather than modifying UI or map-loading logic directly.

4.1.5 How to Integrate a New VPS / Map Partner

If a future developer needs to connect a different VPS or map provider, the preferred approach is to keep the existing application flow unchanged and introduce a new implementation of `IExternalMapService`.

Recommended integration steps:

1. Create a new class implementing `IExternalMapService`.
2. Implement `AuthenticateAsync(...)` using the partner's authentication model.
3. Implement `GetMapsAsync(...)` and map the provider response into `ExternalMapInfo`.
4. Implement `ResolveMapDownloadUrlAsync(...)` to convert provider-specific asset identifiers into a downloadable URL.
5. Assign the new implementation to `mapServiceSource` in the Unity Inspector.

This preserves compatibility with:

- `CustomUIManager`
- `MapQuery`
- map download and caching
- editor entry flow

In other words, the provider-specific logic should remain isolated inside the new service class. The rest of the system should continue to consume normalized `ExternalMapInfo` data only.

4.1.6 Integration Checklist for Future Developers

When adding a new VPS provider, verify the following:

Item	Description
Authentication	Can credentials be exchanged successfully?
Map list normalization	Are provider fields mapped into <code>ExternalMapInfo</code> correctly?
Download URL resolution	Does the provider return a direct GLB URL or require a second lookup?
Naming strategy	Are <code>mapName</code> and <code>mapCode</code> stable enough for UI display and storage matching?
Asset compatibility	Is the returned mesh format compatible with the existing GLB loading path?

4.1.7 Example Integration Pattern

The intended pattern is:

```

public class PartnerMapService : MonoBehaviour, IExternalMapService
{
    public bool IsAuthenticated { get; private set; }

    public Task AuthenticateAsync(string clientId, string clientSecret)
    {
        // Partner-specific auth
    }

    public Task<IReadOnlyList<ExternalMapInfo>> GetMapsAsync(CancelationToken)
    {
        // Convert partner map objects into ExternalMapInfo
    }

    public Task<string> ResolveMapDownloadUrlAsync(ExternalMapInfo info)
    {
        // Return final downloadable GLB URL
    }
}

```

This is the preferred extension point and should be treated as the contract boundary for VPS integration.

4.2 UI Workflow Controller

Source file:

- Assets/Script/VPS/CustomUIManager.cs

4.2.1 Responsibility

CustomUIManager is the primary workflow controller for the front-end experience before the editor phase begins.

It manages:

- authentication UI
- map retrieval UI
- map preview and selection
- transition into edit mode

4.2.2 Main Behavior

The module:

1. resolves the configured map service

2. authenticates using client credentials
3. fetches the available map list
4. populates the map dropdown
5. triggers map download via MapQuery
6. updates camera framing for the loaded map
7. transitions from preview mode into editing mode

4.2.3 Dependencies

- IExternalMapService
- MapQuery
- CameraManager
- Unity UI controls and panels

4.2.4 Notes

CustomUIManager currently combines UI orchestration and workflow control in a single class. This is acceptable for current scope, but future growth may justify splitting state handling from direct UI manipulation.

Suggested figure placeholders

Figure 4-1. Authentication and map selection UI

Figure 4-2. Preview-to-edit transition flow

4.3 Map Retrieval, Caching, and Database Import

Source files:

- Assets/Script/VPS/MapQuery.cs
- Assets/Script/VPS/MapStorage.cs

4.3.1 Responsibility

MapQuery is the integration hub for:

- remote map download
- local map caching
- GLB loading
- Firebase-based marker dataset discovery
- Firebase-based marker dataset import into the editor

4.3.2 Map Download Flow

For a selected map:

1. the external provider resolves a download URL
2. MapQuery checks the local cache
3. if cached, the local binary is loaded directly
4. if not cached, the GLB is downloaded
5. the file is optionally written into local cache
6. the map is instantiated using GLTFast

4.3.3 Cache Strategy

Map files are cached under Unity persistent storage using either:

- a sanitized mapCode, or
- a SHA-256 hash of the mesh URL

This approach avoids repeated network downloads and improves repeated-load performance.

4.3.4 Firebase Import Flow

The database import flow is:

1. query /Marks.json
2. inspect saved entries and match them against the current mapName
3. populate the Load DB dropdown with matching saved titles
4. when the user selects a title, request /Marks/{title}.json
5. convert JSON into Dictionary<string, MarkStorage.MarkData>
6. pass the dictionary to EditorManager.LoadMarksFromData(...)

4.3.5 MapStorage

MapStorage is a static in-memory registry for normalized map metadata. It is used primarily by the selection and download flow.

4.3.6 Operational Considerations

- databaseUrl must be configured correctly in the Unity Inspector.
- Firebase import behavior depends on map metadata matching the currently loaded map.
- cache clearing is supported globally and by specific mapCode.

Suggested figure placeholders

Figure 4-3. Map download and cache flow

Figure 4-4. Load DB workflow

4.4 Editor Core

Source files:

- `Assets/Script/CustomEditor/EditorManager.cs`
- `Assets/Script/CustomEditor/MarkerService.cs`
- `Assets/Script/CustomEditor/PanelController.cs`
- `Assets/Script/CustomEditor/MarkController.cs`
- `Assets/Script/CustomEditor/MarkStorage.cs`

4.4.1 Responsibility

The editor subsystem provides the marker authoring workflow and acts as the runtime coordination point for editing, simulation, and upload phases.

`EditorManager` is the central entry point and delegates responsibility to supporting services.

4.4.2 Editor Modes

The system defines the following modes:

1. None
2. Marking
3. Setting
4. Scripting
5. Simulation
6. Upload

Each mode controls both:

- UI visibility and workflow progression
- permitted user interaction behavior

4.4.3 Subcomponents

MarkerService

Responsible for marker-related authoring logic:

- drag-to-create marker regions
- marker object creation and destruction
- marker label management
- marker selection and highlighting
- geometry and directional visualization
- import reconstruction from saved marker datasets

PanelController

Responsible for editor UI mode transitions:

- initialize and hide panels
- switch visible panels by mode
- prepare upload summary information
- trigger upload and return-to-menu actions

MarkController

Attached to created marker objects and forwards click-based selection back to EditorManager.

MarkStorage

Static in-memory storage for authored marker data, keyed by label.

4.4.4 Marker Data Model

Each marker stores:

- label
- position
- scale
- margin
- directional angles
- keyword
- details

This combines spatial configuration with semantic metadata for downstream simulation and AI guidance.

Suggested figure placeholders

Figure 4-5. Marker authoring UI

Figure 4-6. Setting mode visualization

Figure 4-7. Scripting mode metadata editing

4.5 Simulation Subsystem

Source files:

- Assets/Script/CustomEditor/SimulationController.cs
- Assets/Script/CustomEditor/ToolManager.cs

4.5.1 Responsibility

The simulation subsystem evaluates the simulated user's spatial relationship to authored markers and exposes that context to the UI and AI agent layer.

4.5.2 SimulationController

Key responsibilities:

- activate and manage a simulated player marker
- support mouse-based and keyboard-based control schemes
- determine the nearest valid marker relative to the simulated user
- compute whether the user is:
 - not associated with any marker
 - near a marker
 - inside a marker
- compute user-relative direction to the marker
- write the result into UI output fields used elsewhere in the system

4.5.3 State Outputs

The simulation writes to:

- nearestOutputText
- directionOutputText
- insideOutputText
- simpleMSGText
- currentSituation

These values are then consumed by the AI agent workflow.

4.5.4 Evaluation Logic

Marker evaluation is based on:

1. rectangular region inclusion
2. additional margin
3. directional angle filtering
4. nearest-center selection when multiple markers qualify
5. inside-vs-near classification

4.5.5 ToolManager

Provides simulation-adjacent utility behavior:

- right-click preview display
- preview camera positioning
- simulated heading control via left/right input

- camera zoom support through Cinemachine framing

Suggested figure placeholders

Figure 4-8. Simulation mode UI

Figure 4-9. Direction and marker-state evaluation diagram

4.6 AI Guidance Subsystem

Source files:

- `Assets/Script/Agent/AgentController.cs`
- `Assets/Script/Agent/AgentApiSettings.cs`
- `Assets/Script/Agent/IAgentTextService.cs`
- `Assets/Script/Agent/IAgentSpeechService.cs`
- `Assets/Script/Agent/OpenAITextService.cs`
- `Assets/Script/Agent/LocalSpeechService.cs`
- `Assets/Script/Agent/OpenAITtsSpeechService.cs`
- `Assets/Script/Agent/FallbackSpeechService.cs`
- `Assets/Script/Agent/AgentJsonUtility.cs`

4.6.1 Responsibility

The AI guidance subsystem listens to simulation state and produces short guidance text plus spoken output when the simulated user context stabilizes around a marker.

4.6.2 Architectural Structure

This subsystem is intentionally split into:

- orchestration: `AgentController`
- configuration: `AgentApiSettings`
- text generation abstraction: `IAgentTextService`
- speech abstraction: `IAgentSpeechService`
- concrete provider implementations

This significantly improves future provider portability.

4.6.3 AgentController

The controller does not determine spatial state itself. Instead, it:

1. observes `EditorManager` simulation outputs
2. detects stable state transitions
3. builds a prompt using:
 - current simulation message

- target marker label
 - marker keywords
 - marker details
4. requests generated text from the configured text service
 5. requests speech playback from the configured speech service

4.6.4 Trigger Rules

Speech generation only occurs when:

1. the system is in `Simulation` mode
2. `currentSituation` is not `None`
3. the current label/situation pair remains stable for `stableSeconds`
4. the same label/situation pair has not already been handled
5. the cooldown window has elapsed

4.6.5 Speech Strategy

Current implementation uses a fallback chain:

1. attempt local Windows speech synthesis
2. if unavailable, fall back to OpenAI TTS

4.6.6 Engineering Note

The project has already moved away from hard-coded provider usage inside `AgentController`. This is a strong foundation for future multi-provider support.

4.6.7 AI Interface-Based Integration Strategy

The AI subsystem now exposes two explicit extension points:

- `IAgentTextService`
- `IAgentSpeechService`

This means future developers should not replace AI logic by editing `AgentController` directly. Instead, they should add or swap service implementations while preserving the existing trigger and prompt orchestration flow.

4.6.8 How to Integrate a New Text Provider

To integrate another LLM or agent text API:

1. Create a new class implementing `IAgentTextService`.
2. Implement `GenerateText(...)` using the target provider's request/response contract.
3. Normalize the provider response into a plain string result.

4. Instantiate the new service in `AgentController.InitializeServices()`, or in a future provider factory.

Example:

```
public class GeminiTextService : IAgentTextService
{
    public IEnumerator GenerateText(string prompt, Action<string> on
    {
        // Provider-specific request
        // Parse response
        // Return normalized text
    }
}
```

Because `AgentController` only expects generated text, the rest of the simulation-to-guidance workflow can remain unchanged.

4.6.9 How to Integrate a New Speech Provider

To integrate another TTS platform:

1. Create a new class implementing `IAgentSpeechService`.
2. Implement `Speak(...)` for the target TTS API or local speech engine.
3. Return success or error through the existing callback pattern.
4. Replace or chain the service in `AgentController.InitializeServices()`.

Example:

```
public class AzureSpeechService : IAgentSpeechService
{
    public IEnumerator Speak(string text, AudioSource audioSource, /
    {
        // Provider-specific synthesis and playback
    }
}
```

This allows the project to support:

- a single direct TTS provider
- a local-first / cloud-fallback strategy
- multiple speech backends selected by environment or deployment target

4.6.10 Recommended AI Provider Integration Rules

For maintainability, future developers should follow these rules:

1. Keep simulation trigger logic inside `AgentController`.
2. Keep provider-specific HTTP and parsing logic inside service implementations.
3. Keep configuration values inside `AgentApiSettings` or a future provider-specific settings object.
4. Avoid embedding provider-specific request code directly in scene controllers.
5. Treat `IAgentTextService` and `IAgentSpeechService` as the primary substitution boundaries.

4.6.11 Future Improvement Recommendation

The current implementation already supports interface-based substitution, but future maintainability would improve further with a provider-selection factory, for example:

```
public static class AgentServiceFactory
{
    public static IAgentTextService CreateTextService(AgentProvider provider)
    public static IAgentSpeechService CreateSpeechService(AgentProvider provider)
}
```

This would allow developers to switch providers through configuration instead of code changes.

Suggested figure placeholders
Figure 4-10. AI guidance trigger flow
Figure 4-11. AI configuration diagram

4.7 Firebase Upload Subsystem

Source file:

- `Assets/Script/Firebase/UploadDB.cs`

4.7.1 Responsibility

`UploadDB` serializes the current marker dataset from `MarkStorage` and uploads it into Firebase Realtime Database.

4.7.2 Upload Modes

The class supports two upload modes:

1. `UploadAllMarks()`
Writes the current marker set directly to `/Marks.json`

2. UploadMark(title)

Writes the current marker set under /Marks/{title}.json

4.7.3 Serialized Fields

Each marker entry includes:

- position
- scale
- margin
- angle1
- angle2
- keyword
- details

If the currently selected map can be resolved from UI state, a map field is also included in the dataset root.

4.7.4 Role in the Overall System

The upload subsystem closes the authoring loop by making marker datasets reusable across sessions and reloadable through the MapQuery import path.

Suggested figure placeholders

Figure 4-12. Upload workflow

Figure 4-13. Firebase data structure example

5. End-to-End Workflow

5.1 Map Acquisition Workflow

1. User enters platform credentials.
2. The system authenticates through the configured IExternalMapService.
3. Available maps are requested and displayed.
4. The user selects a map.
5. The selected map is downloaded or loaded from cache.
6. The loaded map is instantiated into Unity and shown in preview.

5.2 Authoring Workflow

1. User enters edit mode.
2. In Marking mode, the user drags to define a marker region.
3. The user assigns a label.
4. In Setting mode, the user configures margin and directional angles.

5. In Scripting mode, the user enters semantic content such as keywords and details.

5.3 Simulation and AI Workflow

1. User switches to Simulation mode.
2. The simulated player is controlled using mouse or keyboard input.
3. The system continuously evaluates the nearest relevant marker.
4. Once the state becomes stable, AgentController constructs a prompt.
5. AI text is generated.
6. Speech playback is attempted through the configured speech service chain.

5.4 Save and Reload Workflow

1. User switches to Upload mode.
2. User provides a dataset title.
3. Marker data is uploaded to Firebase.
4. On future sessions, MapQuery loads saved titles matching the current map.
5. A selected dataset is imported back into the editor.

6. Data Model

6.1 ExternalMapInfo

Field	Description
id	Provider-specific map identifier
mapName	Human-readable map name
mapCode	Map code or alternate identifier
thumbnailUrl	Preview thumbnail URL
storageInMb	Approximate map size
createdAt	Map creation timestamp
meshLink	Direct mesh URL or provider file reference

6.2 MarkStorage.MarkData

Field	Description
label	Marker label

position	World position
scale	Marker dimensions
margin	Additional trigger margin
angle1	Start angle for directional filtering
angle2	End angle for directional filtering
keyword	Short semantic tag(s)
details	Long-form semantic description

6.3 Firebase Example Structure

```
{
  "Marks": {
    "DemoTitle": {
      "map": "Example Map",
      "Lobby": {
        "position": { "x": 0, "y": 0, "z": 0 },
        "scale": { "x": 2, "y": 3, "z": 2 },
        "margin": 1.5,
        "angle1": -30,
        "angle2": 30,
        "keyword": "entrance",
        "details": "Main lobby area"
      }
    }
  }
}
```

7. Configuration

7.1 Map Provider Configuration

Map service configuration is defined by the active `IExternalMapService` implementation. Typical configurable items include:

- provider base URL
- authentication method
- map list endpoint
- download URL resolution strategy

7.2 Firebase Configuration

Firebase configuration is now centralized in:

- `Assets/Script/Firebase/FirebaseConfig.cs`

The shared entry point is:

- `FirebaseConfig.DatabaseUrl`

This value is used by both:

- `MapQuery` for Firebase reads
- `UploadDB` for Firebase writes

This reduces maintenance risk and helps ensure that upload and import operations always point to the same Firebase environment.

7.3 AI Configuration

`AgentApiSettings` currently centralizes:

- API key
- text model
- text endpoint base URL
- system prompt
- token and temperature settings
- language code
- speech model
- speech endpoint base URL
- voice
- audio format
- sample rate

8. Extension Strategy

8.1 Adding a New Map Platform

To integrate a new platform:

1. create a new class implementing `IExternalMapService`
2. map provider-specific responses into `ExternalMapInfo`
3. implement URL resolution logic
4. assign the new service in the relevant Unity Inspector field

This should not require changes to CustomUIManager or MapQuery beyond configuration.

8.2 Adding a New AI Provider

To integrate a new AI platform:

1. implement IAgentTextService for the desired text API
2. optionally implement IAgentSpeechService for the target TTS platform
3. wire the implementation into AgentController initialization

Examples of future provider classes include:

- GeminiTextService
- ClaudeTextService
- AzureSpeechService
- ElevenLabsSpeechService

8.3 Documentation and Delivery Improvement

For production delivery, the following additions are recommended:

- scene composition diagrams
- Inspector configuration screenshots
- Firebase dataset examples
- simulation state examples
- sequence diagrams for the AI trigger path

9. Appendix

9.1 Key Source Files

- CustomUIManager.cs
- MapQuery.cs
- MapStorage.cs
- IExternalMapService.cs
- MultiSetMapService.cs
- ExampleMapService.cs
- EditorManager.cs
- MarkerService.cs
- PanelController.cs
- SimulationController.cs
- ToolManager.cs
- MarkController.cs

- `MarkStorage.cs`
- `UploadDB.cs`
- `AgentController.cs`
- `AgentApiSettings.cs`
- `OpenAITextService.cs`
- `OpenAITtsSpeechService.cs`
- `LocalSpeechService.cs`
- `FallbackSpeechService.cs`

10. Conclusion

`JJ_MultiSet` currently provides a coherent end-to-end workflow spanning map acquisition, marker authoring, simulation, persistence, and AI guidance generation.

From a software engineering perspective, the project already has useful modularity in two critical areas: external map provider integration and AI provider integration. With continued improvements in configuration centralization, validation, and documentation, the system is well positioned for maintainability and future extension.